UPSSA
UNIVERSITÉ PRIVÉE DE LA SANTÉ ET DES SCIENCES D'AGADIR
الجامعة الخاصة للصحة و العلوم لأكادير
ⵜⴰⵙⴷⴰⵡⵜ ⵜⵉⵖⵓⵔⵉⵡⵜ ⵏ ⵜⵎⵙⵓⵙⵜ ⴷ ⵜⵜⵓⵖⵔⵉⵡⵉⵏ ⵏ ⵓⴳⴰⴷⵉⵔ

ÉCOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE D'AGADIR
المدرسة العليا الخاصة للهندسة بأكادير
ⵜⵉⵍⵍⵉ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵜⵉⵖⵓⵔⵉⵡⵜ ⵏ ⵓⵜ ⵍⵍⴰⵏ ⴰⴳⴰⴷⵉⵔ

Système d'exploitation Linux

Pr. Salwa BELAQZIZ
s.belaqziz@uiz.ac.ma

Année universitaire 2026-2027

Chapitre 2 · Utilisateurs et permissions

Identités et comptes, gestion des comptes, su et sudo, contrôle d'accès, droits spéciaux et ACL

Plan du chapitre

1
Identités et comptes
UID et GID, /etc/passwd, /etc/shadow, /etc/group, connexion

2
Gérer comptes et groupes
useradd, usermod, passwd, chage, groupadd, cycle de vie

3
Élever ses droits
su, sudo et les règles de /etc/sudoers

4
Contrôler l'accès
chown, chgrp, ordre des tests, droits des répertoires, umask

5
Aller plus loin
SUID, SGID, sticky bit, ACL, chattr, diagnostic

Prérequis du chapitre 1 : lire une ligne de `ls -l`, changer les droits avec `chmod` (symbolique et octal), principe du `umask`. Ces notions ne sont pas reprises ici.

Chaque action est faite au nom d'un compte, identifié par un UID

Le noyau doit savoir **au nom de qui** chaque action est faite pour l'autoriser ou la refuser : il compare l'identité du processus aux droits du fichier.

Utilisateur
Identifié par un **UID**. Possède des fichiers, lance des processus.

→

Groupe
Identifié par un **GID**. Regroupe des utilisateurs qui partagent des droits.

→

Processus
Hérite de l'UID et des GID de celui qui l'a lancé.

→

Fichier
Appartient à un utilisateur et à un groupe.

UID	Type de compte	Exemples
`0`	Super-utilisateur : tous les droits	`root`
`1 – 999`	Comptes système : font tourner les services	`www-data, sshd`
`≥ 1000`	Utilisateurs humains : se connectent	`alice, bob`

Le noyau ne manipule que les **numéros**. Le nom n'est qu'une étiquette lue dans `/etc/passwd`.

root (UID 0) échappe à presque tous les contrôles : on travaille en compte normal et on n'utilise root que pour administrer.

/etc/passwd décrit chaque compte en sept champs

Chaque ligne de `/etc/passwd` décrit un compte, en sept champs séparés par « : ».

ali	x	1001	1001	Ali Salmi	/home/ali	/bin/bash
1 · Identifiant	2 · Mot de passe → /etc/shadow	3 · UID	4 · GID principal	5 · Nom complet (GECOS)	6 · Répertoire personnel	7 · Shell de connexion

Compte	UID	Répertoire	Shell
root	0	/root	/bin/bash
www-data	33	/var/www	/usr/sbin/nologin
mysql	110	/var/lib/mysql	/bin/false
ali	1001	/home/ali	/bin/bash

Shell de connexion

`/bin/bash` : connexion possible.

`nologin` ou `/bin/false` : connexion refusée. C'est le cas des **comptes système**, qui font tourner un service avec le minimum de droits.

```
$ getent passwd ali
$ sudo useradd -r -s /usr/sbin/nologin monapp
```

Lisible par tous (644) : il ne contient aucun mot de passe. On le modifie avec `usermod` ou `vipw`.

/etc/shadow garde les mots de passe hachés, /etc/group décrit les groupes

/etc/shadow · mots de passe hachés (root seul)

```
bob:$y$j9T$Kq…:20454:1:90:7:14:20819:
```

Champ	Signification
1	Identifiant
2	Mot de passe haché (détail ci-dessous)
3	Dernier changement, en jours depuis le 1/1/1970
4 · 5 · 6	Délai mini, durée maxi, préavis (jours)
7 · 8	Délai d'inactivité, expiration du compte

Le champ 2 se lit en trois parties : **algorithme**, **sel**, **hachage**

`$6$`	`Kx9vL2pQ$`	`mT8zQ1rV…`
Algorithme	Sel aléatoire	Hachage

`$1$` MD5 obsolète · `$6$` SHA-512 · `$y$` yescrypt (défaut récent) · `!` en tête : compte verrouillé

/etc/group · les groupes

```
dev:x:1500:alice,bob
```

Champ	Signification
1	Nom du groupe
2	Mot de passe de groupe (rare)
3	GID
4	Membres secondaires

Le groupe **principal** est donné par le champ 4 de `/etc/passwd`, pas par cette liste.

Le sel rend différents les hachages de deux mots de passe identiques : impossible de les comparer à une table précalculée. Un champ vide ne demande aucun mot de passe.

À la connexion, Linux compare le hachage saisi à celui de /etc/shadow

Ce qui se passe quand ali tape son identifiant et son mot de passe :

1 — Saisie
login ou l'écran graphique lit identifiant et mot de passe

→

2 — /etc/passwd
retrouve ali : UID 1001, /home/ali, /bin/bash

→

3 — /etc/shadow
lit le sel et le hachage enregistrés

→

4 — Comparaison
hache le mot de passe saisi avec ce sel et compare

→

5 — Session
si égal : prend l'UID d'ali et lance son shell

Le mot de passe n'est jamais stocké en clair

Le système garde seulement une **empreinte à sens unique**. Pour vérifier, il refait le calcul et compare.

Même root ne peut pas relire un mot de passe : il peut seulement le remplacer.

```
$ sudo tail -3 /var/log/auth.log
sshd: Accepted password for ali
sshd: Failed password for fatima
sshd: Failed password for invalid user admin
# invalid user : le compte n'existe pas
```

id et who disent qui l'on est ; last et les journaux disent qui s'est connecté

Qui suis-je, qui est connecté ?

Commande	Affiche
`whoami`	Le nom de l'utilisateur courant
`id [ali]`	UID, GID principal et tous les groupes
`groups`	La liste des groupes
`who · w`	Les sessions ouvertes (w : leur activité)

```
$ id
uid=1001(ali) gid=1001(ali)
groupes=1001(ali),27(sudo),1500(dev)
$ who
ali   pts/0   2026-10-10 09:12
```

Dans `id`, **gid=** donne le groupe principal et **groupes=** liste tous les groupes, principal compris.

Qui s'est connecté, et quand ?

Commande · fichier	Ce qu'on y apprend
`last -n 5 ali`	Les 5 dernières sessions d'ali
`lastb`	Les tentatives échouées (root)
`lastlog`	Dernière connexion de chaque compte
`faillock --user ali`	Échecs comptés, blocage
`/var/log/auth.log`	Connexions, su, sudo (Debian)
`/var/log/secure`	Même rôle (Red Hat)

```
$ sudo grep 'Failed password' /var/log/auth.log | wc -l
412
```

Un compte utilisé à une heure ou depuis un lieu inhabituels doit alerter : consultez ces journaux régulièrement.

useradd -m crée un compte complet en cinq opérations

Ce que fait `sudo useradd -m -s /bin/bash alice`, étape par étape :

1 **Choisit l'UID**
le premier libre ≥ UID_MIN (1000)

→

2 **Écrit /etc/passwd**
ajoute la ligne d'ali

→

3 **Écrit /etc/shadow**
mot de passe « ! » : compte verrouillé

→

4 **Crée le groupe privé**
alice:x:1001: dans /etc/group

→

5 **Crée /home/ali**
copie le modèle /etc/skel

Les valeurs par défaut viennent de trois emplacements

Emplacement	Ce qu'il fixe
`/etc/login.defs`	Plages d'UID, durée de vie des mots de passe, UMASK
`/etc/default/useradd`	Shell, répertoire de base, expiration
`/etc/skel/`	Modèle copié dans chaque nouveau /home

`useradd -D` affiche souvent `/bin/sh` : pensez à `-s /bin/bash`.

Groupe privé

Chaque utilisateur reçoit un groupe du même nom, dont il est le seul membre : ses fichiers ne sont partagés avec personne par défaut.

Tant que `passwd ali` n'a pas été lancé, personne ne peut se connecter. Modifier les défauts ne touche que les comptes créés **ensuite**.

usermod modifie un compte, userdel le supprime

Commande · option	Effet
`useradd -m`	Crée aussi le répertoire personnel
`-s /bin/bash · -c "Ali Ahmed"`	Shell · nom complet
`-g dev · -G web,sudo`	Groupe principal · secondaires
`-u 1050 · -e 2027-06-30`	UID imposé · date d'expiration
`usermod -aG dev ali`	Ajoute ali au groupe dev
`usermod -l fafa fatima`	Renomme le compte
`usermod -d /data/ali -m ali`	Déplace le répertoire personnel
`userdel ali`	Supprime le compte, garde /home
`userdel -r ali`	Supprime le compte et son /home

```
$ sudo useradd -m -s /bin/bash \
    -c "Ali Ahmed" -G dev ali
$ sudo passwd ali
$ sudo usermod -aG sudo ali
```

Le piège de -G : `usermod -G web bob` **remplace** la liste des groupes secondaires. Ali perd sudo et dev.

Toujours écrire `usermod -aG` (a = append).

Sur Debian et Ubuntu, `adduser ali` fait la même chose en mode interactif.

passwd change le mot de passe, chage règle sa durée de vie

Commande	Effet
`passwd`	Change son propre mot de passe
`passwd ali`	root change celui de ali
`passwd -e ali`	Force un changement à la prochaine connexion
`passwd -S ali`	Affiche l'état du mot de passe
`chage -l ali`	Affiche les règles d'expiration
`chage -M 90 ali`	Mot de passe valable 90 jours au plus
`chage -m 1 -W 7 ali`	1 jour mini entre deux changements, préavis 7 jours
`chage -E 2027-01-01 ali`	Le compte expire à cette date

```
$ sudo chage -l ali
Dernier changement   : janv. 01, 2026
Mot de passe expire  : avril 01, 2026
Mot de passe inactif : avril 15, 2026
Le compte expire     : janv. 01, 2027
Nb max. de jours     : 90
Avertissement        : 7
```

Expiré ou inactif ?

Mot de passe expiré : bob se connecte encore, mais doit en choisir un nouveau.

Compte inactif ou expiré : la connexion est refusée ; seul root peut le réactiver.

Verrouiller un mot de passe ne bloque pas toutes les connexions

Bloquer un compte peut viser le mot de passe, le shell ou le compte entier. Le bon choix dépend de ce qui doit rester possible

Commande	Ce qu'elle change	Clé SSH	Annuler
`passwd -l ali`	Ajoute ! devant le hachage	Encore possible	`passwd -u ali`
`usermod -L ali`	Identique à passwd -l	Encore possible	`usermod -U ali`
`usermod -s /usr/sbin/nologin ali`	Remplace le shell de connexion	Bloquée	`usermod -s /bin/bash ali`
`chage -E 0 ali`	Fait expirer le compte	Bloquée	`chage -E -1 ali`

```
$ sudo passwd -S ali
ali L 2026-01-01 1 90 7 14   # L = verrouillé
```

Le groupe principal sert à créer, les groupes secondaires à accéder

Groupe principal

Un seul, champ 4 de `/etc/passwd`.

Sert à **créer** : chaque nouveau fichier prend ce groupe.

Groupes secondaires

Zéro ou plusieurs, listés dans `/etc/group`.

Servent à **accéder** : ils comptent comme le principal.

```
$ id ali
uid=1002(ali) gid=1002(ali) groupes=1002(ali),1500(dev)
$ touch rapport.txt ; ls -l rapport.txt
-rw-r--r-- 1 ali ali 0 rapport.txt   # ali:ali, pas dev
```

Commande	Effet
`groupadd -g 1500 dev`	Crée dev
`groupmod -n devs dev`	Renomme
`groupdel dev`	Supprime
`gpasswd -a ali dev`	Ajoute ali
`gpasswd -d ali dev`	Retire ali
`gpasswd -A ali dev`	ali gère dev

Obtenir le groupe dev sur un fichier : `chgrp dev` après , `newgrp dev` pour la session, ou le **SGID** sur le répertoire .

Un nouveau membre doit **se reconnecter** (ou lancer `newgrp dev`) pour profiter du groupe.

Un compte vit quatre étapes : arrivée, mutation, absence et départ

1 · Arrivée

Créer le compte, forcer le changement du mot de passe

```
$ useradd -m -G dev ali
$ passwd ali
$ passwd -e ali
```

2 · Mutation

Changer d'équipe sans perdre les autres groupes

```
$ usermod -aG compta ali
$ gpasswd -d ali dev
```

3 · Absence

Bloquer temporairement, puis rétablir

```
$ usermod -L ali
$ usermod -U ali
```

4 · Départ

Bloquer, archiver, puis supprimer

```
$ chage -E 0 ali
$ tar czf ali.tgz /home/ali
$ userdel -r ali
```

Pourquoi archiver avant de supprimer ?

`userdel -r` efface /home/ali, mais les fichiers de lea ailleurs (/srv, /tmp) restent avec un UID devenu **orphelin** : `ls -l` affiche un numéro à la place du nom.

Pour les retrouver : `sudo find / -nouser -o -nogroup 2>/dev/null`

su ouvre une session sous une autre identité, sudo exécute une seule commande

La syntaxe de su a été vue au chapitre 1. Ici, on compare les deux façons d'agir avec d'autres droits.

Critère	su · substitute user	sudo · superuser do
Portée	Un shell complet sous une autre identité	Une seule commande
Mot de passe demandé	Celui du compte cible (root)	Le sien, mémorisé 15 min
Ce qui est permis	Tout ce que peut le compte cible	Seulement ce qu'autorise /etc/sudoers
Traçabilité	On sait qu'on est devenu root, pas ce qu'on a fait	Chaque commande est journalisée avec son auteur

```
$ sudo apt update  # une commande en root
$ sudo -u www-data ls /var/www  # en tant qu'un autre compte
$ sudo -i  # shell root, à éviter
$ sudo -l  # mes droits sudo
$ sudo !!  # relancer la dernière commande avec sudo
```

Une règle sudoers précise qui peut lancer quoi, sous quelle identité

Chaque règle de `/etc/sudoers` répond à quatre questions :

`ali`	`ALL`	`(ALL:ALL)`	`/usr/bin/apt`
Qui ? un utilisateur ou %groupe	**Sur quelle machine ?** ALL = toutes	**En tant que qui ?** utilisateur:groupe cible	**Quelles commandes ?** ALL ou chemins absolus

```
# /etc/sudoers.d/web  (sudo visudo -f …)
%sudo   ALL=(ALL:ALL) ALL      # Debian, Ubuntu
%wheel  ALL=(ALL) ALL          # Red Hat
User_Alias WEBADM = ali, fatima
Cmnd_Alias WEB = /usr/bin/systemctl restart nginx, \
                 /usr/bin/systemctl reload nginx
WEBADM  ALL=(root) WEB
%dev    ALL=(www-data) /usr/bin/git pull
```

Élément	Sens
`User_Alias`	Liste nommée d'utilisateurs
`Cmnd_Alias`	Liste nommée de commandes
`%dev`	Les membres du groupe dev
`NOPASSWD:`	Pas de mot de passe

Toujours éditer avec `visudo`, qui vérifie la syntaxe : une erreur dans sudoers bloque sudo pour tout le monde.

sudo est sûr quand chaque compte n'a que les commandes dont il a besoin

Pratique	Pourquoi
Travailler en compte normal	Une faute de frappe en root peut détruire le système
Préférer sudo à su	Chaque commande est tracée et attribuée
Lister les commandes permises	ALL donne root sans restriction
Écrire des chemins absolus	Un faux apt placé dans le PATH serait accepté
Éviter NOPASSWD	Un poste resté ouvert donne root à qui passe
Une règle par fichier dans sudoers.d	Facile à relire et à retirer

```
$ sudo journalctl \
   _COMM=sudo | tail -1
ali : TTY=pts/0 ;
USER=root ;
COMMAND=/usr/bin/apt
update
```

`sudo vi` ou `sudo bash` ouvre un shell root complet : la règle « une commande précise » est contournée.

Ne donnez jamais un éditeur ou un shell par sudo.

chown change le propriétaire d'un fichier, chgrp change son groupe

Qui a le droit ?

Changer le **propriétaire** : root seulement.

Changer le **groupe** : le propriétaire, vers un groupe dont il est membre.

Commande	Effet
`chown ali f`	ali devient propriétaire
`chown ali:dev f`	Propriétaire et groupe
`chown :dev f`	Groupe seulement
`chgrp dev f`	Groupe seulement
`chown -R ali:dev projet/`	Tout le contenu du répertoire

`chown` et `chgrp` ne touchent pas aux droits : ils changent **à qui** s'appliquent les blocs u et g.

Pour modifier les droits eux-mêmes, on utilise `chmod` (chapitre 1).

Le noyau teste le propriétaire, puis le groupe, puis les autres, et s'arrête au premier qui correspond



Une seule catégorie

La **première** qui correspond s'applique, et le test s'arrête.

Avec `----rw----`, le propriétaire n'a **aucun** droit, même s'il est dans le groupe.

Supprimer un fichier dépend des droits du **répertoire** (w + x), pas de ceux du fichier.

Sur un répertoire, x ouvre le passage et r, w n'agissent qu'avec lui

Le rôle de r, w et x sur un répertoire est vu au chapitre 1. Ce qui compte en pratique, c'est leur **combinaison** :

Droits	ls rep	cd rep	cat rep/f	touch rep/n
`r--`	noms seuls [1]	non	non	non
`--x`	non	oui	oui [2]	non
`r-x`	oui	oui	oui	non
`-wx`	non	oui	oui [2]	oui
`rwx`	oui	oui	oui	oui

[1] ls affiche les noms avec des « ? » à la place des détails. [2] Seulement si l'on connaît le nom exact du fichier.

w sans x ne sert à rien : pour créer ou supprimer dans un répertoire, il faut w et x.

`chmod 711` sur son répertoire personnel : les autres traversent vers un fichier connu, sans pouvoir lister le contenu.

Exercice 1 : Lire une ligne de /etc/passwd

Voici la ligne du compte de ali dans le fichier `/etc/passwd`.

```
# /etc/passwd (une ligne)
ali:x:1002:1002:ali Ahmed:
  /home/bob:/bin/bash
```

Questions

Q1. Quel est le **nom de connexion** de ali et son **UID** ?

Q2. ali est-il un utilisateur humain ou un compte système ?

Q3. Quel est son **répertoire personnel** ?

Q4. ali peut-il ouvrir une session ? Où est rangé son mot de passe ?

Aide

Les 7 champs sont séparés par « : » : identifiant, mot de passe, UID, GID, commentaire, répertoire, shell.

Exercice 1 : Lire une ligne de /etc/passwd

Q1. Nom : ali · UID : 1002

Le 1er champ est le nom de connexion, le 3eme champ est l'UID.

Q2. Un utilisateur humain

Son UID vaut 1002, donc au moins 1000. Les comptes système ont un UID de 1 à 999.

Q3. /home/ali

C'est le 6e champ : le répertoire où ali arrive quand il se connecte.

Q4. Oui · mot de passe dans /etc/shadow

Son shell (7eme champ) est /bin/bash, un vrai shell. Le x indique que le mot de passe haché est dans /etc/shadow, lisible par root seul.

À retenir

Le shell indique si la connexion est possible : /bin/bash l'autorise, /usr/sbin/nologin la refuse.

Exercice 2 : Lire la sortie de la commande id

Ali tape la commande `id` dans son terminal et obtient le résultat ci-contre. Le système est une Ubuntu.

Questions

Q1. Quel est l'**UID** d'Ali ?

Q2. Quel est son **groupe principal** ?

Q3. Quels sont ses **groupes secondaires** ?

Q4. Ali peut-elle utiliser `sudo` ? Pourquoi ?

```
$ id
uid=1001(ali) gid=1001(ali)
  groupes=1001(ali),27(sudo),
  1500(dev)
```

Aide

gid= donne le groupe principal. groupes= liste tous les groupes, le principal compris.

Exercice 2 : Lire la sortie de la commande id

Q1. UID : 1001

On le lit après uid= : 1001, avec le nom ali entre parenthèses.

Q2. Groupe principal : ali (GID 1001)

C'est la valeur après gid=. Chaque utilisateur reçoit un groupe privé du même nom.

Q3. Groupes secondaires : sudo et dev

Ce sont les groupes de la liste groupes=, sauf le groupe principal alice.

Q4. Oui

Sur Debian et Ubuntu, les membres du groupe sudo peuvent lancer des commandes en root avec sudo. Sur Red Hat, c'est le groupe wheel.

À retenir

id résume l'identité d'un utilisateur : son UID, son groupe principal et tous ses groupes.

Exercice 3 : Créer un compte utilisateur

Une nouvelle étudiante, Sara, arrive. Vous êtes administrateur et devez lui créer un compte.

Questions

Q1. Écrivez la commande qui crée le compte **sara** avec un répertoire personnel et le shell `/bin/bash`.

Q2. Quelle commande lui donne un **mot de passe** ?

Q3. Comment **vérifier** que le compte existe ?

Q4. Que se passe-t-il si on oublie de donner un mot de passe ?

```
# options utiles de useradd
-m   crée le répertoire personnel
-s   choisit le shell
-c   nom complet
```

Aide

Les commandes d'administration se lancent avec sudo.

Exercice 3 : Créer un compte utilisateur

Q1. sudo useradd -m -s /bin/bash sara

-m crée /home/sara et y copie les fichiers de /etc/skel. -s /bin/bash choisit le shell de connexion.

Q2. sudo passwd sara

root saisit deux fois le nouveau mot de passe. Il est enregistré haché dans /etc/shadow.

Q3. getent passwd sara ou id sara

Si la commande affiche une ligne pour sara, le compte existe.

Q4. Personne ne peut se connecter

Sans mot de passe, le compte est verrouillé : /etc/shadow contient « ! » à la place du hachage.

```
$ sudo useradd -m -s /bin/bash sara
$ sudo passwd sara
Nouveau mot de passe :
$ id sara
uid=1003(sara) gid=1003(sara)
  groupes=1003(sara)
```

À retenir

Créer un compte se fait en deux temps : useradd crée le compte, passwd l'active.

Exercice 4 : Ajouter un utilisateur à un groupe

Fatima est déjà membre du groupe **dev**. Elle doit aussi entrer dans le groupe **web**, sans quitter dev.

Questions

Q1. Écrivez la commande qui ajoute Fatima au groupe **web**.

Q2. Que se passe-t-il si on oublie l'option `-a` ?

Q3. Comment **vérifier** les groupes de Fatima ?

Q4. À quel moment Fatima profite-t-il de son nouveau groupe ?

```
$ groups Fatima
Fatima dev
```

Aide

-G donne la liste des groupes secondaires. -a veut dire « ajouter » à la liste.

Exercice 4 : Ajouter un utilisateur à un groupe

Q1. sudo usermod -aG web Fatima

-aG ajoute web à la liste des groupes de Fatima et garde les groupes qu'il avait déjà.

Q2. Fatima perd le groupe dev

Sans -a, -G remplace toute la liste : Fatima n'a plus que web. On peut aussi utiliser gpasswd -a Fatima web.

Q3. groups Fatima ou id Fatima

La liste doit maintenant contenir dev et web.

Q4. À sa prochaine connexion

Les groupes sont lus à l'ouverture de session. Fatima se déconnecte et se reconnecte, ou tape newgrp web.

```
$ sudo usermod -aG web fatima
$ groups fatima
fatima dev web
  # avec -G seul (erreur) :
$ sudo usermod -G web fatima
$ groups fatima
fatima web
```

À retenir

Toujours écrire usermod -aG : le -a évite d'effacer les autres groupes.

Exercice 5 : Choisir entre su et sudo

Ali est membre du groupe sudo. Il tape les deux commandes ci-contre.

```
$ sudo apt update
$ su -
```

Questions

Q1. Quel **mot de passe** demande `sudo apt update` ?

Q2. Quel mot de passe demande `su -` ?

Q3. Laquelle des deux méthodes **enregistre** chaque commande dans le journal ?

Q4. Comment Ali revient-il à son compte après `su -` ?

Aide

sudo lance une seule commande. su ouvre une session complète sous une autre identité.

Exercice 5 : Choisir entre su et sudo

Q1. Le mot de passe d'Ali

sudo vérifie qu'Ali est bien Ali, puis consulte /etc/sudoers pour savoir s'il a le droit.

Q2. Le mot de passe de root

su ouvre une session root : il faut connaître le mot de passe du compte cible.

Q3. sudo

Chaque commande lancée avec sudo est enregistrée avec le nom d'Ali. Avec su, on sait seulement qu'une session root a été ouverte.

Q4. Avec la commande exit

exit ferme la session root et ramène Ali dans son propre shell.

```
$ su -
Mot de passe : (celui de root)
# whoami
root
# exit
$ whoami
ali
```

À retenir

On préfère sudo : on ne partage pas le mot de passe root et chaque action est tracée.

Exercice 6 : Qui peut lire ce fichier ?

Fatima est propriétaire de notes.txt. Ali est membre du groupe **dev**. Ahmed n'est pas dans dev.

Questions

Q1. Que peut faire **Fatima** sur ce fichier ?

Q2. Que peut faire **Ali** ?

Q3. Que peut faire **Ahmed** ?

Q4. Ali peut-il **modifier** le fichier ?

```
$ ls -l notes.txt
-rw-r----- 1 fatima dev 900
  notes.txt
```

Aide

Le système teste d'abord le propriétaire, puis le groupe, puis les autres. Il s'arrête au premier cas qui correspond.

Exercice 6 : Qui peut lire ce fichier ?

Q1. Fatima : lire et modifier

Alice est la propriétaire, donc le 1er bloc s'applique : rw-.

Q2. Ali : lire seulement

Bob n'est pas propriétaire mais il est membre de dev : le 2eme bloc s'applique, r--.

Q3. Ahmed : rien

Ahmed n'est ni propriétaire ni membre de dev : le 3eme bloc s'applique, ---.

Q4. Non

Le bloc du groupe ne contient pas w. Pour l'autoriser, Alice tape chmod g+w notes.txt.

```
-rw-r----- 1 fatima dev notes.txt
  # rw-  propriétaire (fatima)
  # r--  groupe dev (ali)
  # ---  autres (ahmed)
```

À retenir

Une seule catégorie s'applique à chaque personne : la première qui correspond.